

Chromium for Android — Architecture Orientation

A reading path and mental model for understanding Chromium on Android: the main blocks, the main classes, how they interact, and where Blink fits.

Source links. Paths below link into github.com/obeletski/chromium on the `digitclassifier` branch. Ambiguous basenames, build output under `out/`, and anything under `third_party/dawn/` are left unlinked — dawn is a git submodule, so its files are not in this repository.

Every path below was verified to exist in this checkout at `main` @ Aug 2026. Written 2026-08-28.

Start with these three, in this order

Doc	Why
docs/source_tree_overview.md	42 lines. The whole directory map. Five minutes.
content/README.md	The <code>content</code> vs <code>chrome</code> split — the single most important boundary. Includes <code>content/architecture.png</code> .
third_party/blink/renderer/README.md	Blink's internal structure.

The mental model: two independent axes

These get conflated, which is why Chromium feels confusing at first.

Axis 1 — layering (compile-time, one-directional)

<code>chrome/</code>	product opinions: Android browser UI, settings, features
<code>components/</code>	reusable across embedders (<code>chrome</code> , <code>webview</code> , <code>ios</code>)
<code>content/</code>	the multi-process sandboxed web platform
<code>base/ net/ mojo/</code>	foundations

Dependencies flow one way. Putting code in the wrong layer is the most common review-blocking mistake.

Axis 2 — processes (runtime)

Browser process	privileged; owns UI, network, disk. On Android this IS the Android app process.
Renderer process	sandboxed; one per site-instance. Blink + V8 live here.
GPU process	the only process that talks to the driver.
Utility / network	services

Blink is a *layer* (`third_party/blink`) that runs in a *process* (renderer). `content` is the thing that owns and orchestrates those processes.

Blink's role, specifically

Blink is the **web platform implementation**: HTML parsing, the DOM tree, CSS, layout, and every JavaScript-visible API. If a spec exists on `chromestatus`, its implementation is in Blink.

Blink is not responsible for:

- networking — `//net`, driven from the browser process
- process and sandbox management — `//content`
- compositing and rasterization — `//cc`
- any UI

V8 is also separate (`//v8`). Blink's `bindings/` directory is the glue that exposes C++ DOM objects to JavaScript.

Inside `third_party/blink/renderer/`

Directory	Contents
<code>core/</code>	The essence: DOM, CSS, layout. Monolithic for historical reasons.
<code>modules/</code>	Self-contained features factored out of core. <code>digitclassifier</code> lives here.
<code>platform/</code>	Lower-level things core depends on: <code>wtf</code> (containers), <code>scheduler</code> .
<code>bindings/</code>	V8-heavy code, kept separate because V8 APIs are security-sensitive.

Blink is a **hard boundary**. Inside it you use WTF types (`blink::Vector`, `blink::String`) and Oilpan GC (`Member<>`, `WeakMember<>`, `Persistent<>`) — not STL, not most of `base/`. Outside it you never touch `blink/renderer` at all; you go through `third_party/blink/public/`.

Caveat. [blink/renderer/README.md](#) opens by linking the canonical "*How Blink Works*" document, which is an external Google Doc, not in the repo. It is the best overview available if you can open it.

What is actually different on Android

The part generic Chromium docs will not tell you.

The process model is not fork/exec

Helper processes are Android **Services**, declared in the manifest. The layer-one sandbox is the `android:isolatedProcess` attribute; a seccomp-BPF syscall filter is layer two. The browser process is protected by the ordinary Android app sandbox.

→ [docs/security/android-sandbox.md](#) (short and excellent)

There is a Java UI layer above `content`

No desktop equivalent — desktop uses `//ui/views` instead. The chain:

```
ChromeTabbedActivity    chrome/android/java/src/org/chromium/chrome/browser/ChromeTabbedActivity.java
-> ChromeActivity       chrome/android/java/src/org/chromium/chrome/browser/app/ChromeActivity.java
-> TabImpl              chrome/android/java/src/org/chromium/chrome/browser/tab/TabImpl.java
-> WebContents (Java)   content/public/android/java/src/org/chromium/content_public/browser/
-> CompositorViewHolder chrome/android/java/src/org/chromium/chrome/browser/compositor/CompositorViewHolo
```

`WebContents` is the seam: a Java object wrapping the C++ `content::WebContents`. Everything crosses via **JNI** — which is why you always edit the `.java` and the `.cc / .h` together.

Startup is Java-first

The Android system starts an Activity; Java then loads `libmonochrome.so` and boots native through `ChromeBrowserInitializer / BrowserStartupController`.

- `content/public/android/java/src/org/chromium/content_public/browser/BrowserStartupController.java`
- `chrome/android/java/src/org/chromium/chrome/browser/init/ChromeBrowserInitializer.java`

→ [docs/android_native_libraries.md](#), [docs/speed/startup/android_startup.md](#)

Reading path after the first three

Process & IPC

- [docs/process_model_and_site_isolation.md](#)
- [docs/mojo_and_services.md](#)
- [docs/security/android-sandbox.md](#)
- [docs/security/android-ipc.md](#)

The Android app layer

- [docs/ui/android/overview.md](#)
- [docs/ui/android/mvc_overview.md](#) — Chrome for Android has a house MVC pattern you will see everywhere
- [docs/android_native_libraries.md](#)

Rendering pipeline

- [third_party/blink/renderer/core/README.md](#)
- [third_party/blink/renderer/platform/README.md](#)
- [third_party/blink/public/README.md](#)
- [docs/how_cc_works.md](#)
- [docs/life_of_a_frame.md](#)
- [docs/render_document.md](#)

Cross-cutting — read early and often

- [docs/threading_and_tasks.md](#)

- [docs/callback.md](#)

Nearly all Chromium code is async task-posting. Without these two the rest is unreadable.

[agents/prompts/knowledge_base.md](#) is a *task* routing table ("how do I add a UMA metric"), not an architecture narrative. Useful later, not now.

A shortcut: trace a feature you already own

`navigator.digitclassifier` crosses almost every boundary above. Tracing it end to end teaches more in an hour than the docs will:

```
navigator_digit_classifier.idl      web-facing surface
-> bindings/ (generated V8 glue)
-> digit_classifier.cc              a Blink module
-> DigitClassifier runtime flag     platform/RuntimeEnabledFeatures.md
-> DawnControlClientHolder          blink/renderer/platform/graphics/gpu
-> Mojo / command buffer            renderer -> GPU process hop
-> Dawn -> WGSL -> SPIR-V           the Adreno driver
```

That single path covers IDL, bindings, Blink module layering, the resource bundle, a runtime-enabled feature, and a cross-process hop.

[docs/digitclassifier/WGSL-Compilation-Path.md](#) already documents the tail end.